JENGA: Effective Memory Management for Serving LLM with Heterogeneity

Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon,
Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You,
Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, Ion Stoica

SOSP 2025

Presenter: Seonghun Jung

KAIST

CASYS | KAIST Computer Architecture & System Lab

Contents

- **Introduction**
  - Background
  - Motivation
- **Approach**
  - Key Idea
  - Architecture
- **Evaluation**
  - Methodology
  - Results
- **Conclusion**

LLM Inference

- **Autoregressive inference has two phases and reuses the KV cache.**
  - Prefill: Use parallel computation → High GPU utilization.
  - Decode: Generates tokens one by one → Low GPU utilization.

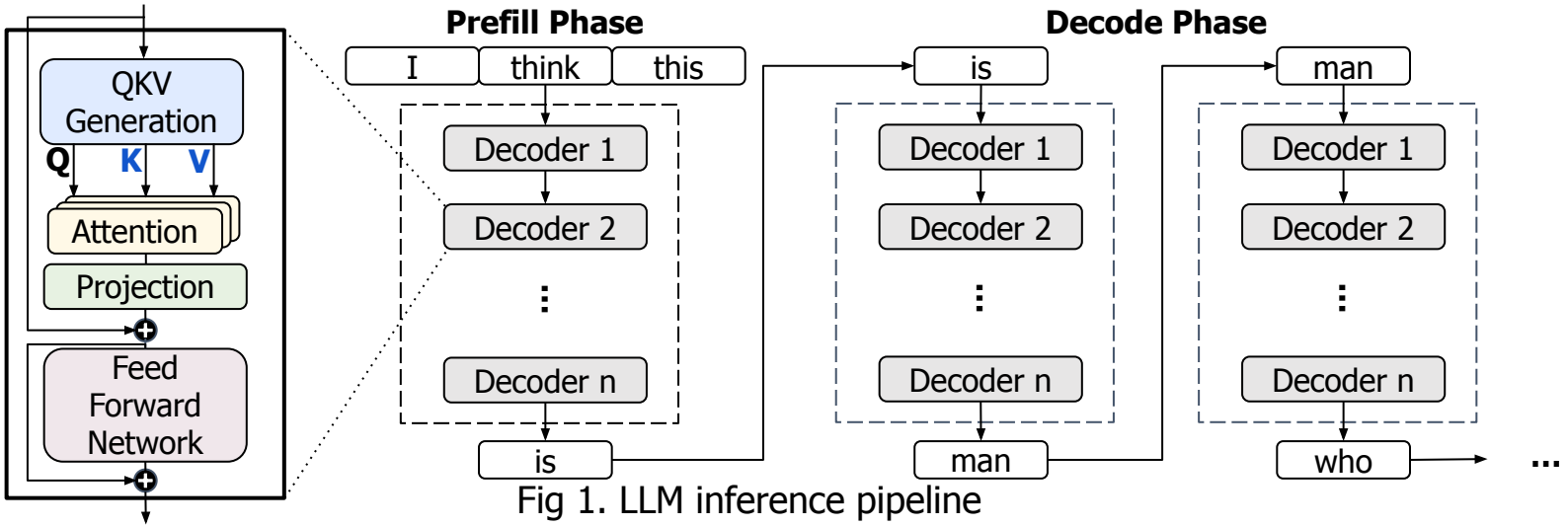


Fig 1. LLM inference pipeline

Maximizing Throughput in LLM Inference

- **Overcoming low GPU utilization through batching**
  - Problem: Sequential token generation → Low GPU utilization → Low throughput & High cost
  - Solution: Batching multiple requests → Enable parallel computation & Boost throughput

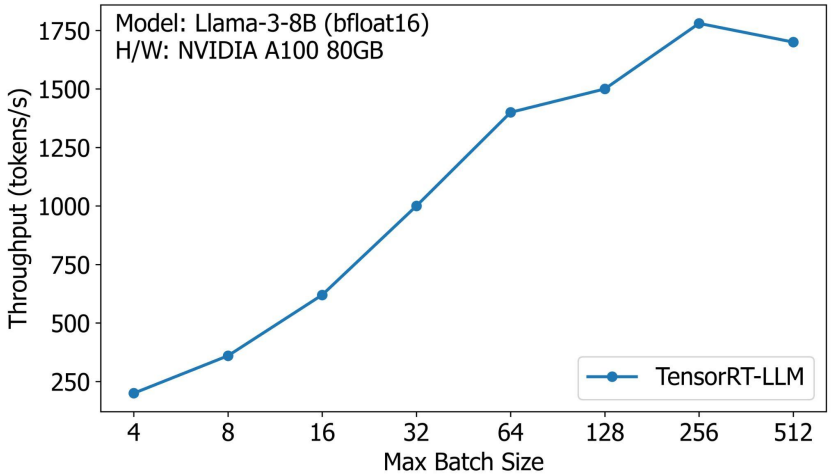


Fig 2. Throughput vs. Batch Size

KV Cache Bottleneck is Severe

- **KV cache waste limits throughput**
  - KV cache grows with sequence length and active requests (batch size).
  - Fragmentation and reservation overhead shrink usable KV memory.

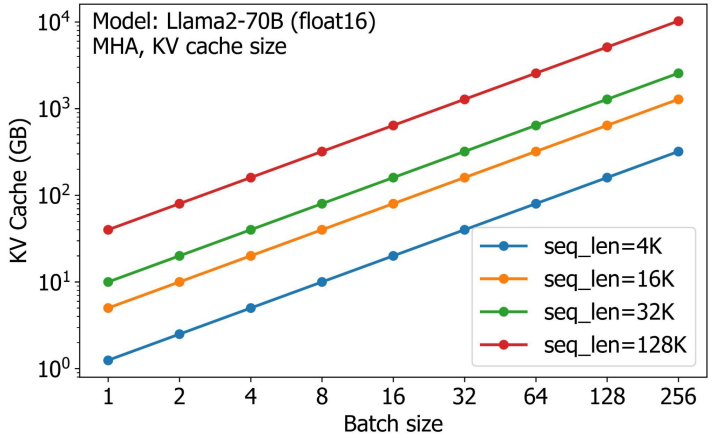


Fig 3. KV cache capacity

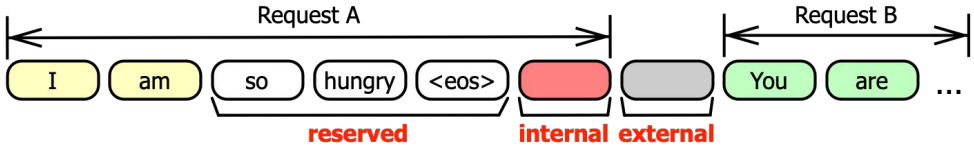


Fig 4. Reserved, internal, external fragmentation

Previous Work: PagedAttention

- **Block-based KV management**
  - Fixed-size paging: Reduced memory fragmentation, achieving higher inference throughput.
  - Prefix caching: Enabled shared-prefix KV cache reuse, reducing redundant computation.

Request prompt: "Four score and seven years ago out"

Output: "fathers" → "brought" → …

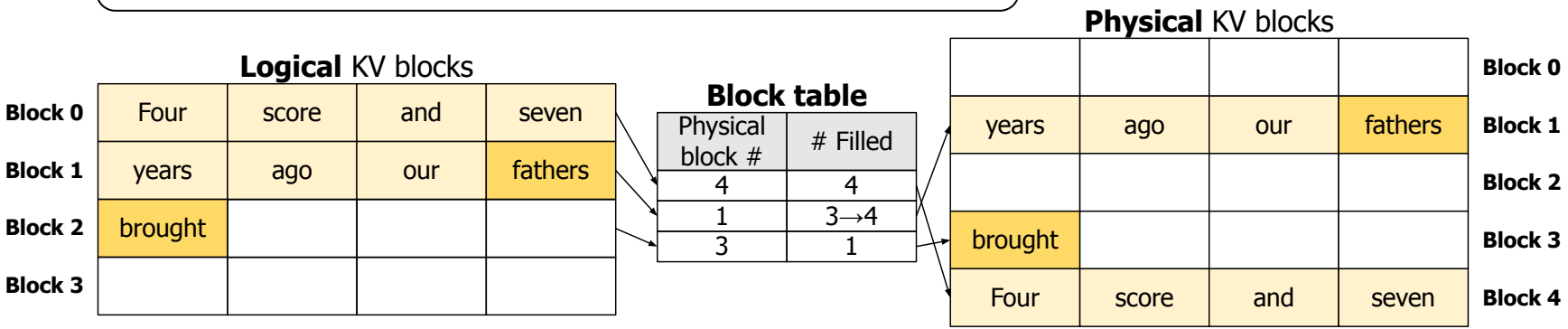


Fig. 5. Block table translation with PagedAttention

Assumptions Behind PagedAttention

- **At the time, monolithic transformer architectures were dominant**
  - Assumption 1: **KV cache size is consistent** across layers/tokens.
  - Assumption 2: Generating the next token **depends on all previous tokens**.

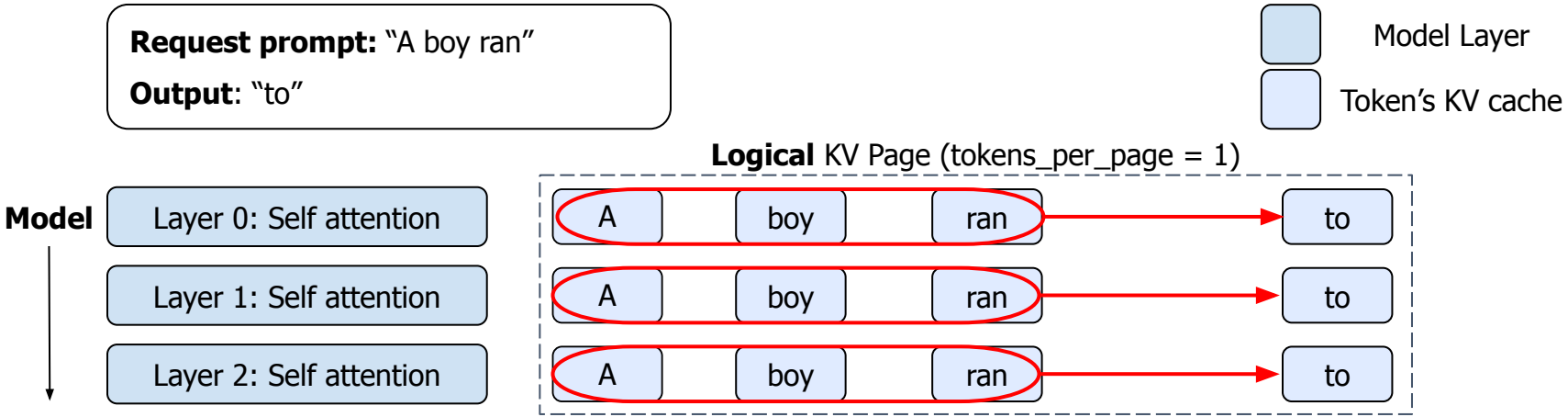


Fig 6. Each layer's logical KV page layout

Assumptions Behind PagedAttention

- **At the time, monolithic transformer architectures were dominant**
  - Assumption 1: **KV cache size is consistent** across layers/tokens.

LLMs are becoming increasingly heterogeneous.

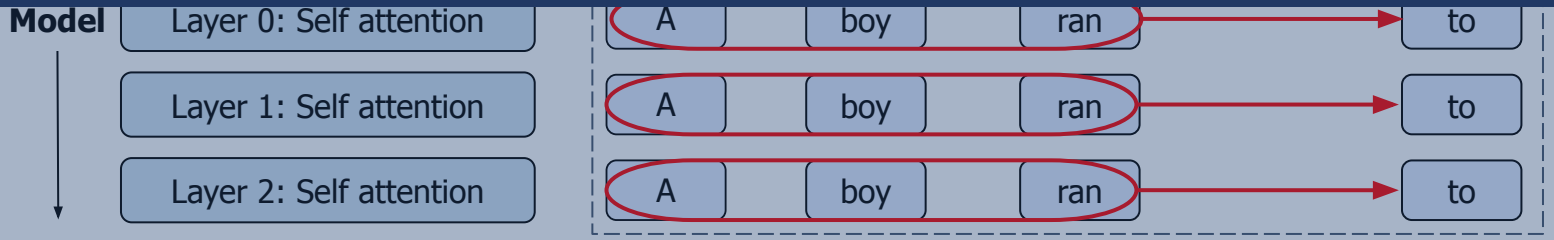


Fig 6. Each layer's logical KV page layout

Multi-modal model

- **e.g. Llama 3.2 vision**
  - Model contains 32 self-attention layers, which require KV cache for text tokens, and 8 cross-attention layers, which require KV cache for image tokens.

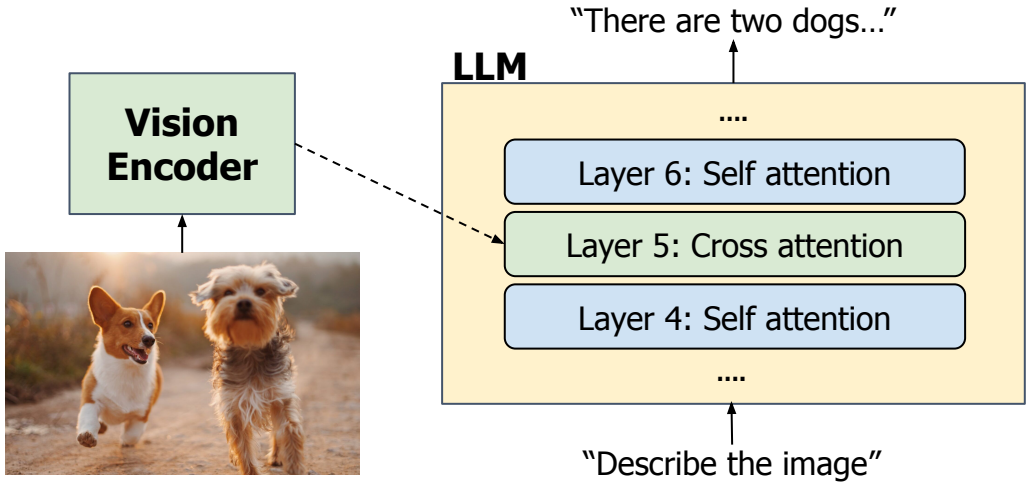


Fig 7. Llama 3.2 vision model architecture

Challenge 1

- **Heterogeneous KV sizes cause memory fragmentation**
  - Since the original paged attention can **only deal with homogeneous layers**, it needs to allocate KV cache for both text and image tokens for all layers.

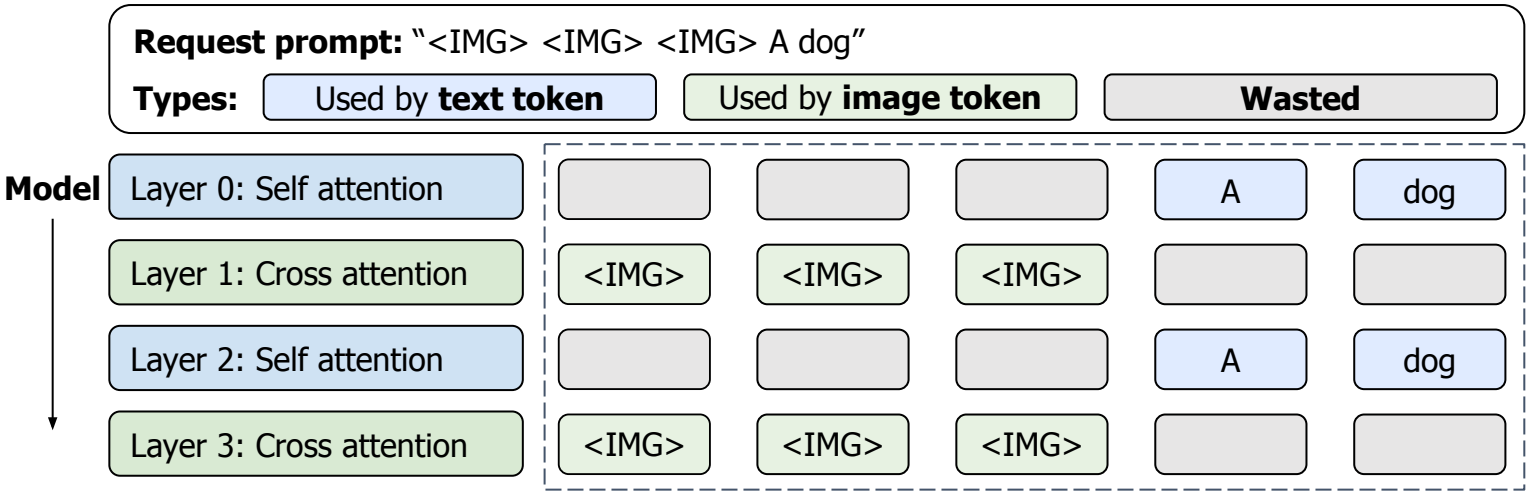


Fig 8. Memory waste in Llama 3.2 Vision (2 cross + 2 self-attention layers)

Sparse attention

- **e.g. Sliding-window attention**
  - Each token only attends to a fixed number of adjacent tokens inside the sliding window.
  - Aim to reduce the KV cache size.

	The	cat	sat	on	the
The	1	0	0	0	0
cat	1	1	0	0	0
sat	1	1	1	0	0
on	1	1	1	1	0
the	1	1	1	1	1

Fig 9. Self attention

	The	cat	sat	on	the
The	1	0	0	0	0
cat	1	1	0	0	0
sat	1	1	1	0	0
on	0	1	1	1	0
the	0	0	1	1	1

Fig 10. Sliding window attention

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

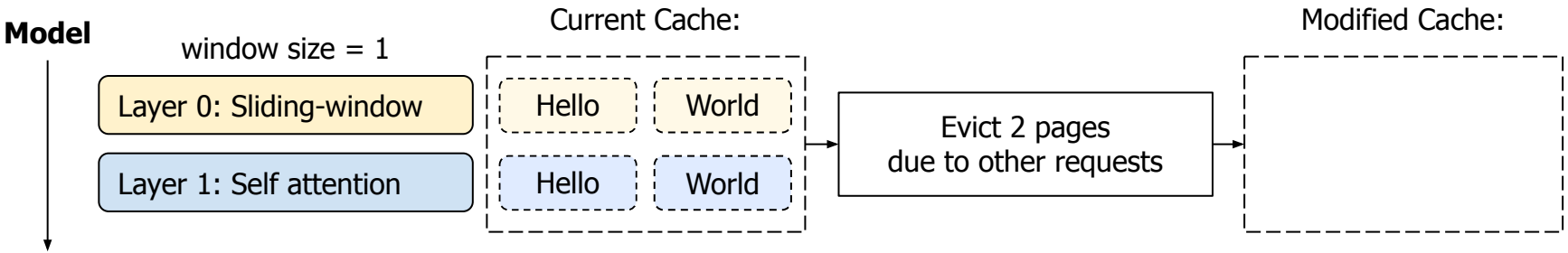


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

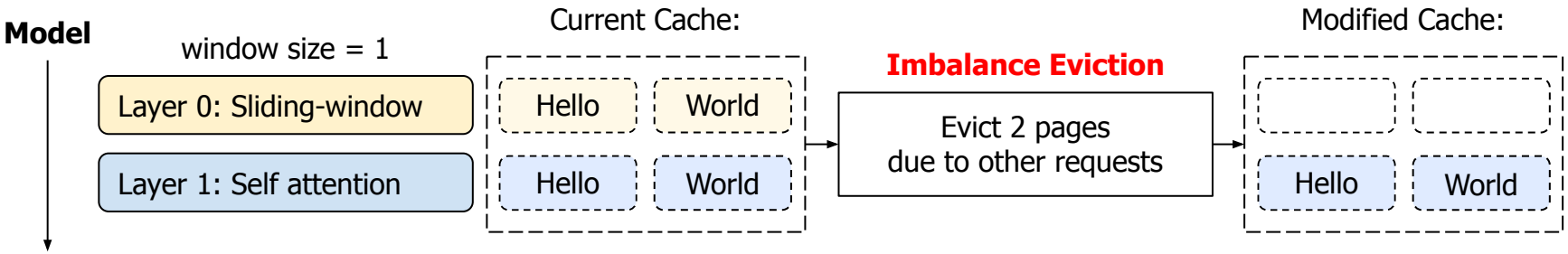


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

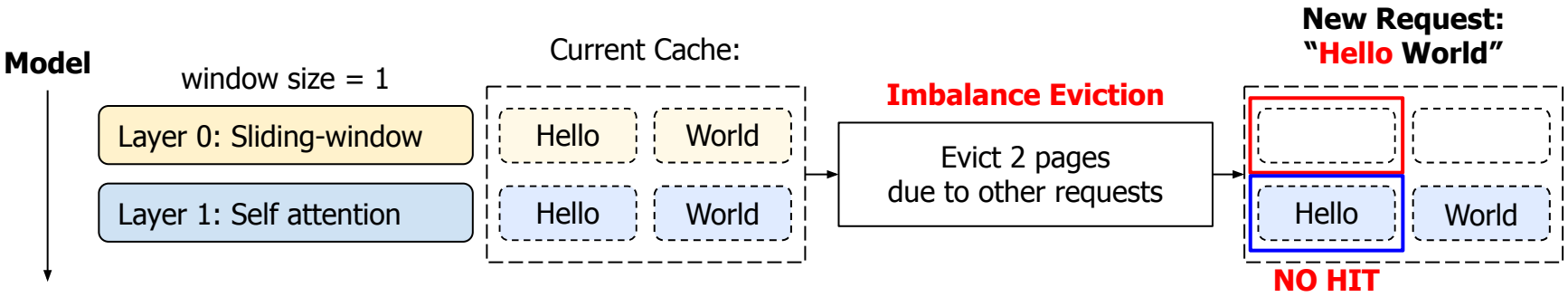


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

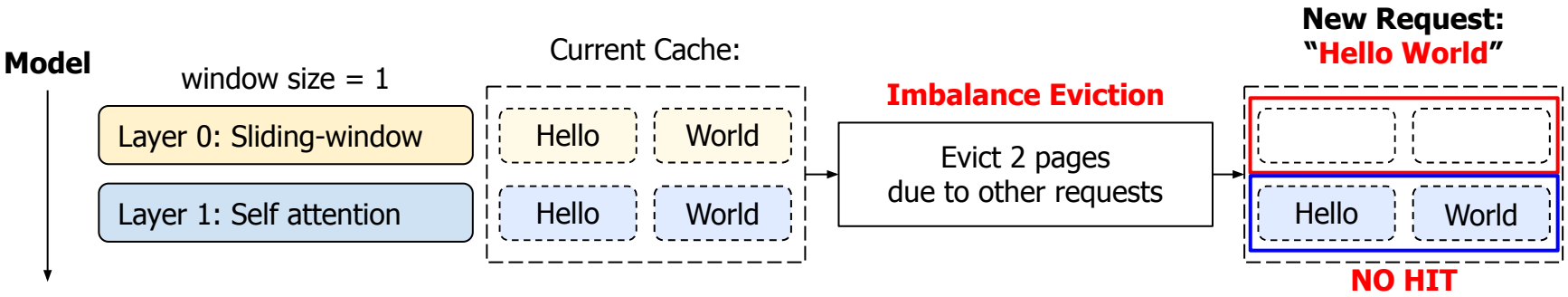


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

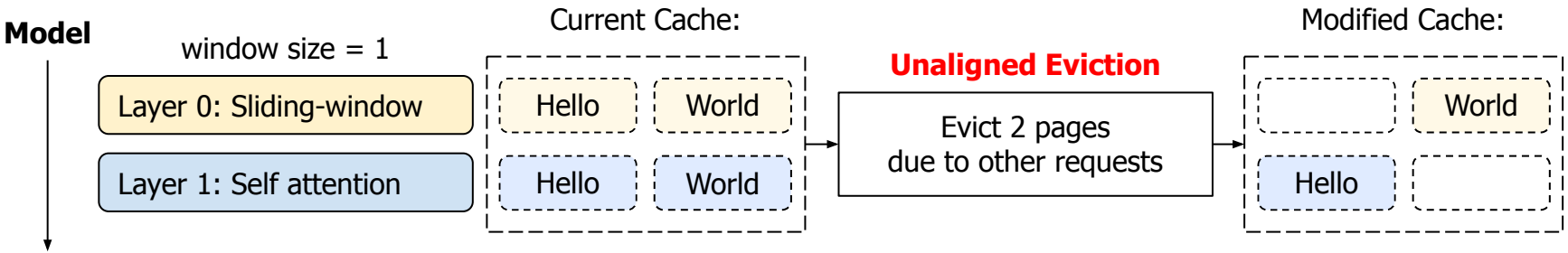


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

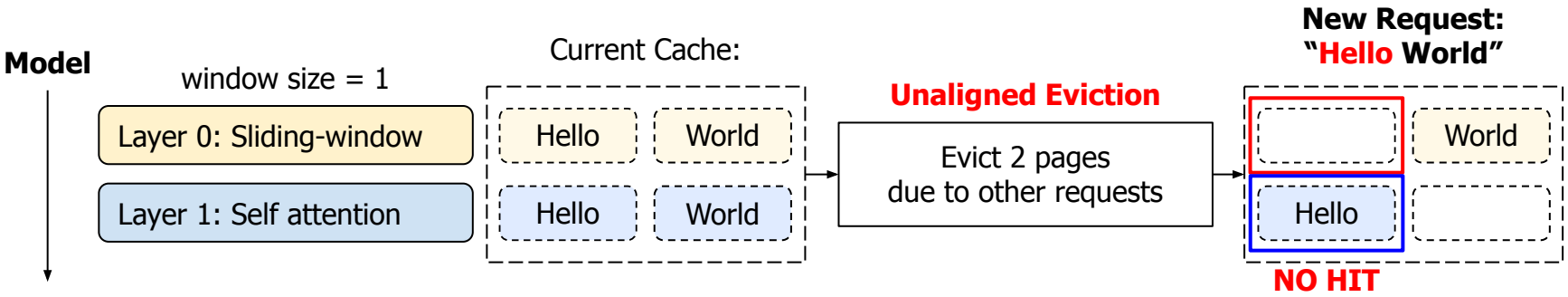


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

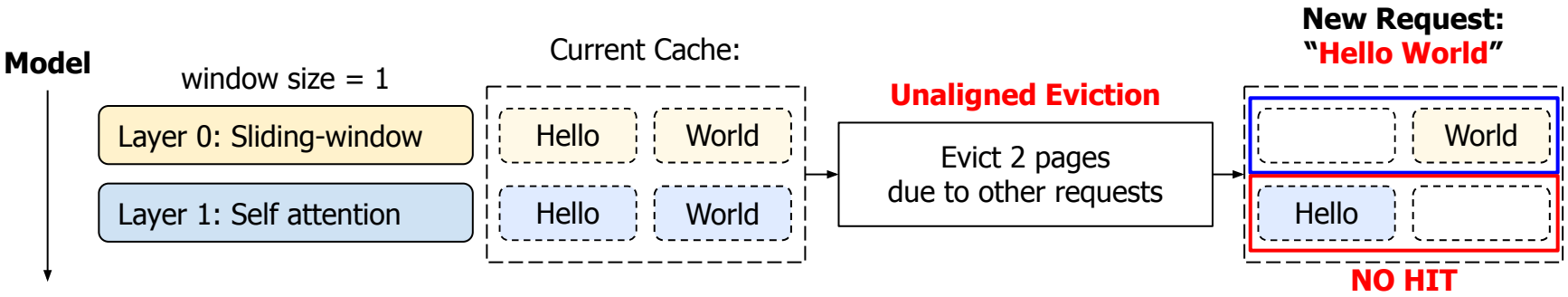


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

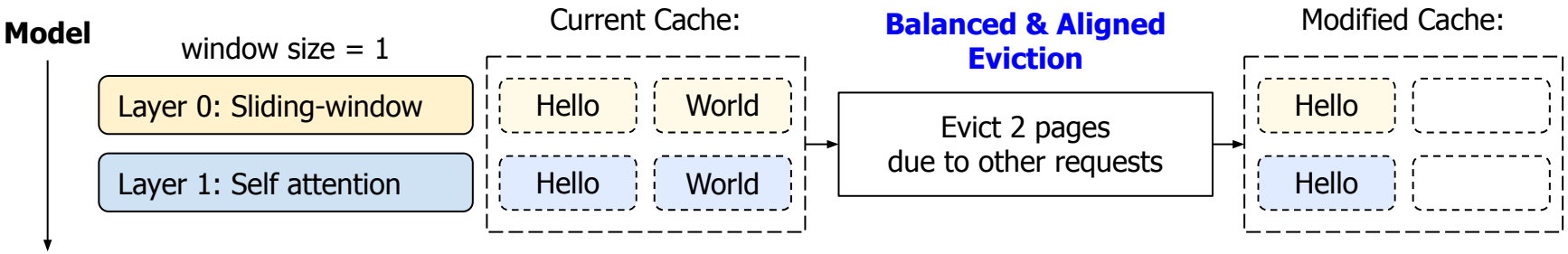


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.
  - Prefix cache hit: Prefix must be **cached in every layer type**.

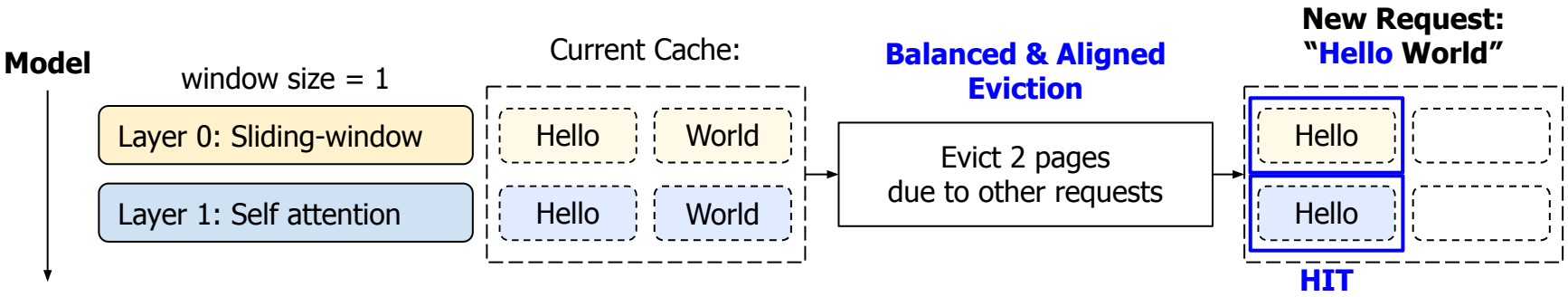


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Challenge 2

- **Heterogeneous Dependency Leads to poor prefix caching**
  - **Single global rule does not work** since access pattern & hit condition differs by layer.

1. **Reduce memory fragmentation**
2. **Maintain balanced, aligned caching across layers**

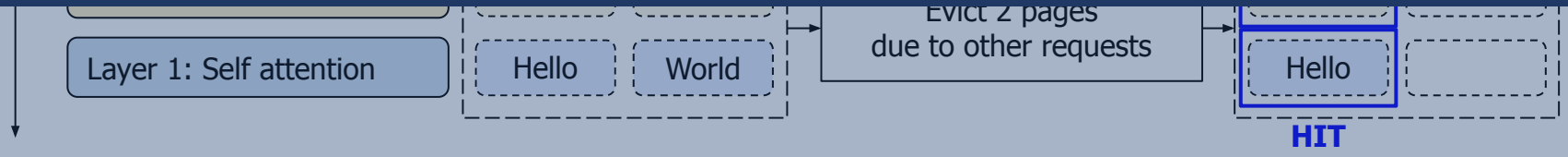


Fig 11. Balanced and aligned cache eviction policy can improve hit rate

Overview of JENGA

- **Two-level memory management system for different types of layers**
  - Reduce memory fragmentation: least common multiple(LCM) & customized allocators.
  - Maintain balanced and aligned caching: prefix-subset & customized evictors.

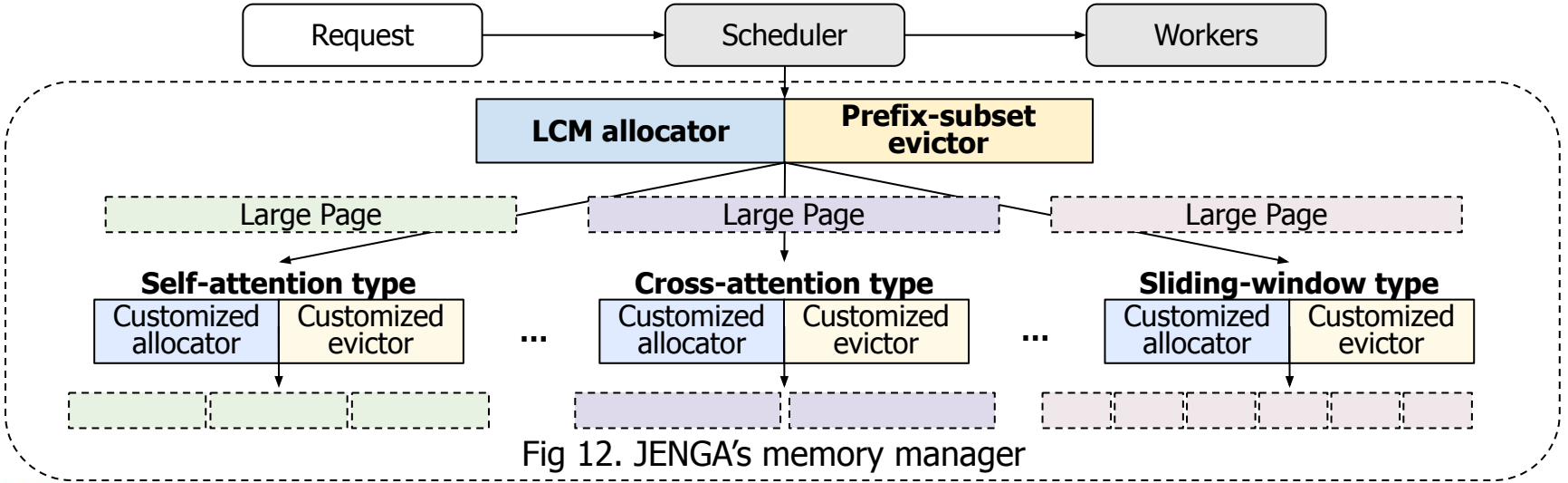


Fig 12. JENGA's memory manager

Overview of JENGA

- **Two-level memory management system for different types of layers**
  - Reduce memory fragmentation: least common multiple(LCM) & customized allocators.
  - Maintain balanced and aligned caching: prefix-subset & customized evictors.

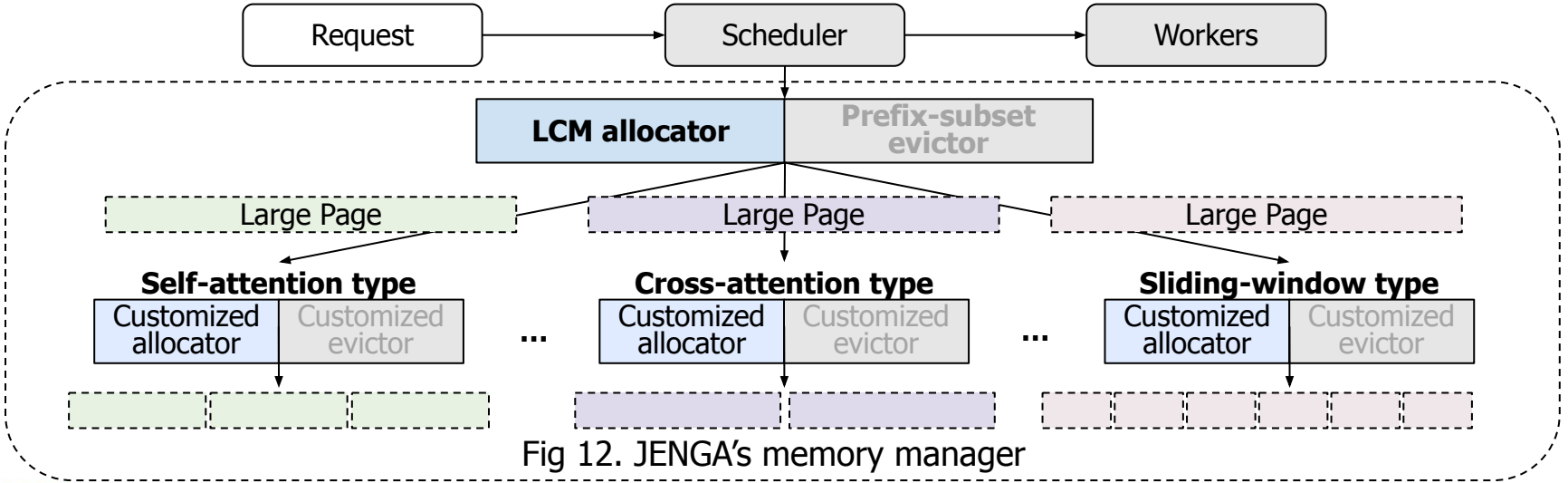


Fig 12. JENGA's memory manager

LCM & Customized Memory Allocator

- **New memory layout for inter-type memory exchange**
  - JENGA sets the **large-page size to the LCM** of layer-type page sizes, then splits it into **type-specific small pages** to prevent **external fragmentation.**

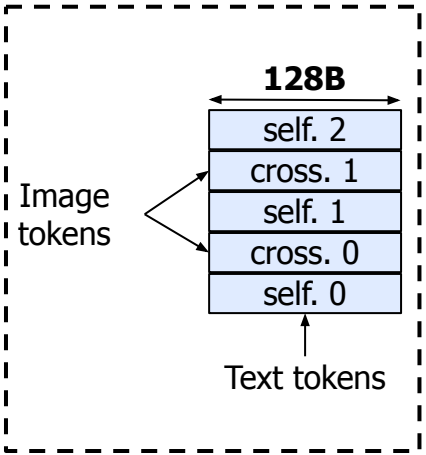


Fig 13. Example model

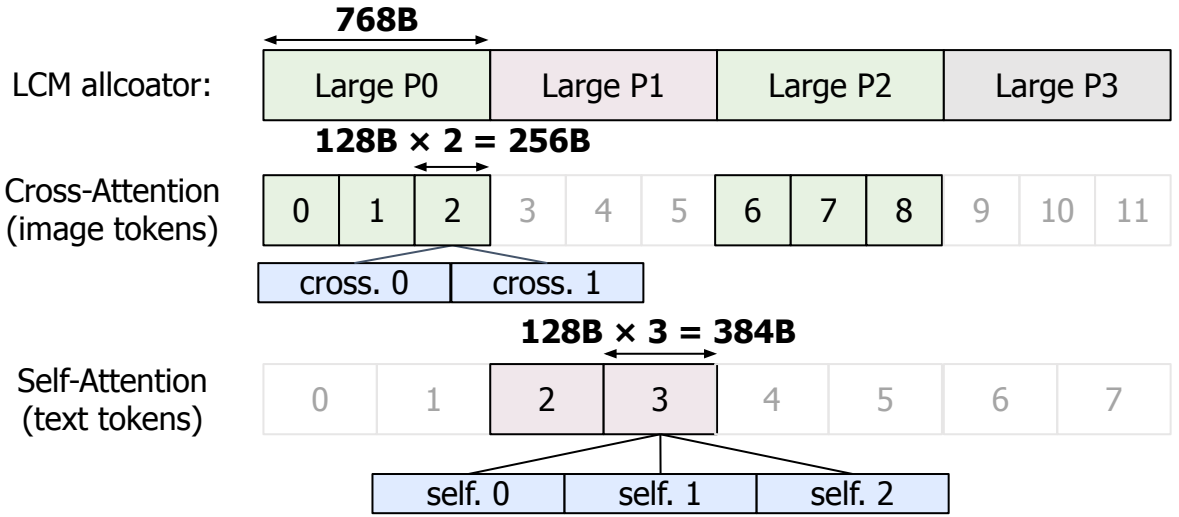


Fig 14. Memory layout of example model

Request-aware Allocation

- **Allocation strategy ➡ Reduce internal fragmentation of large pages**
  - JENGA allocates small pages within **a single large page to the same request** whenever feasible, so large pages can be returned when that request completes.

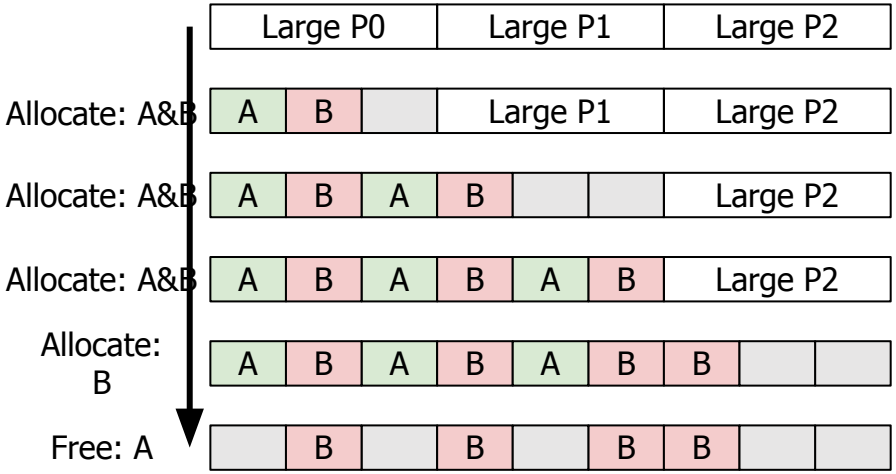


Fig 15. Naïve way with high fragmentation

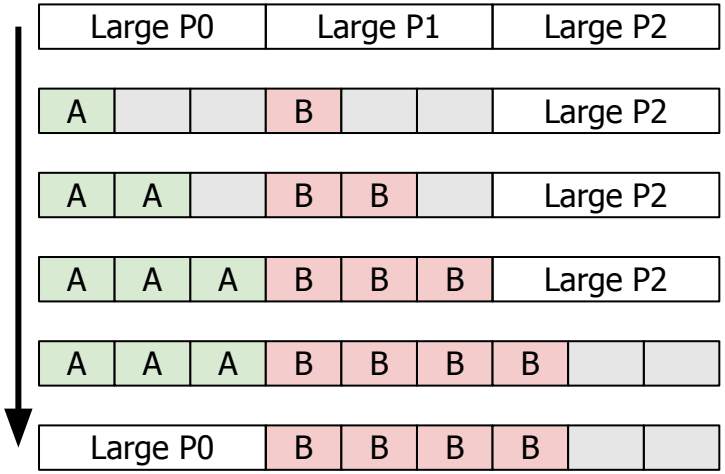


Fig 16. Request-aware with low fragmentation

Customizable Prefix Caching

- **Heterogeneous dependency across layer ➡ Customize & Coordinate**
  - Customizable eviction & hit: Unified per-layer APIs
  - Coordinate manager: Global prefix-subset evictor

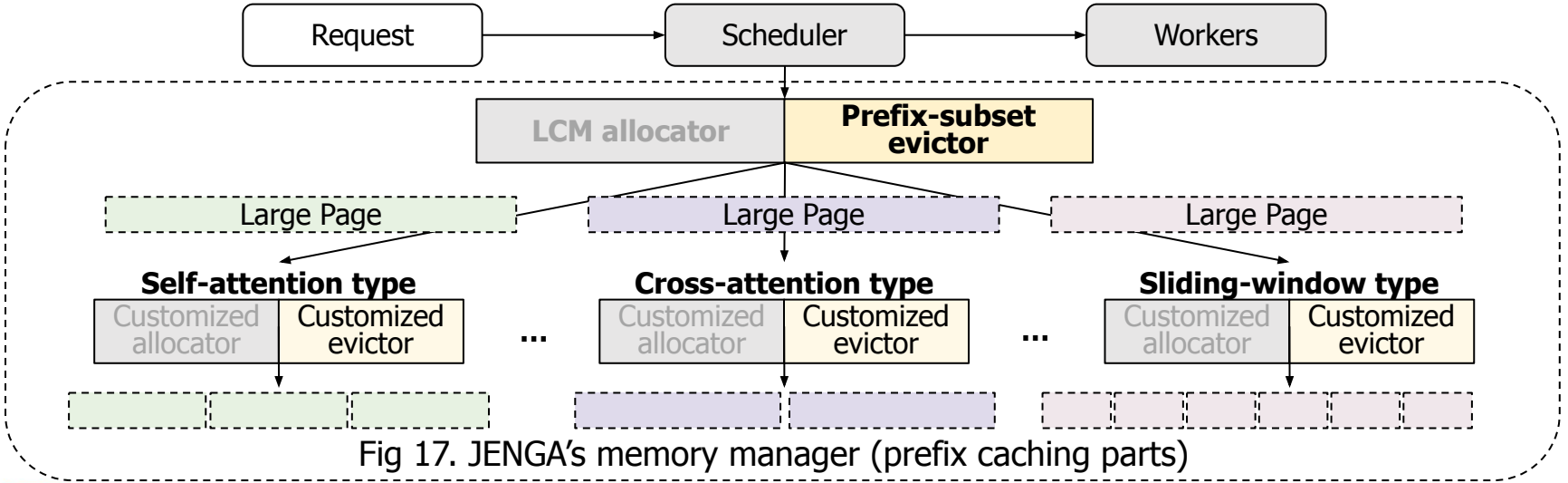


Fig 17. JENGA's memory manager (prefix caching parts)

Customizable Eviction API

- **Balanced, aligned caching across heterogeneous layers**
  - *update_last_access*: Set unifies last-access times across different layers (**balanced eviction**)
  - *set_prefix_length*: Refine eviction priorities for pages (**aligned eviction**)

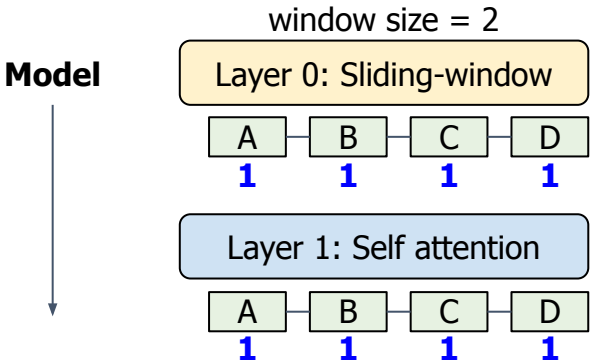


Fig 18. KV cache page with last-access time

Step	Request phase	Sequences
1	1's prefill	ABCD > E
2	1's decode	ABCDE > F
3	2's prefill	ABCDG > H

Fig 19. Timeline of the 2 requests

Customizable Eviction API

- **Balanced, aligned caching across heterogeneous layers**
  - *update_last_access*: Set unifies last-access times across different layers (**balanced eviction**)
  - *set_prefix_length*: Refine eviction priorities for pages (**aligned eviction**)

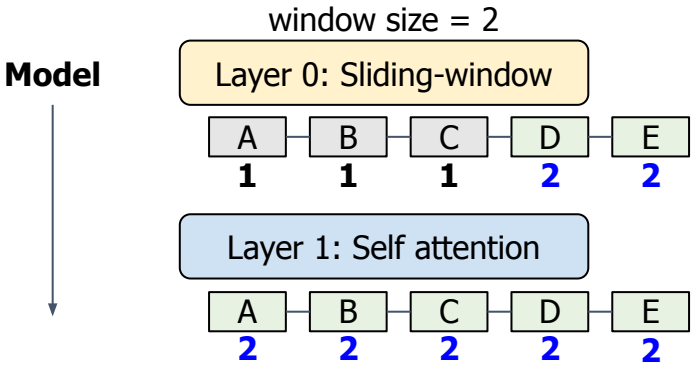


Fig 18. KV cache page with last-access time

Step	Request phase	Sequences
1	1's prefill	ABCD > E
2	1's decode	ABCDE > F
3	2's prefill	ABCDG > H

Fig 19. Timeline of the 2 requests

Customizable Eviction API

- **Balanced, aligned caching across heterogeneous layers**
  - *update_last_access*: Set unifies last-access times across different layers (**balanced eviction**)
  - *set_prefix_length*: Refine eviction priorities for pages (**aligned eviction**)

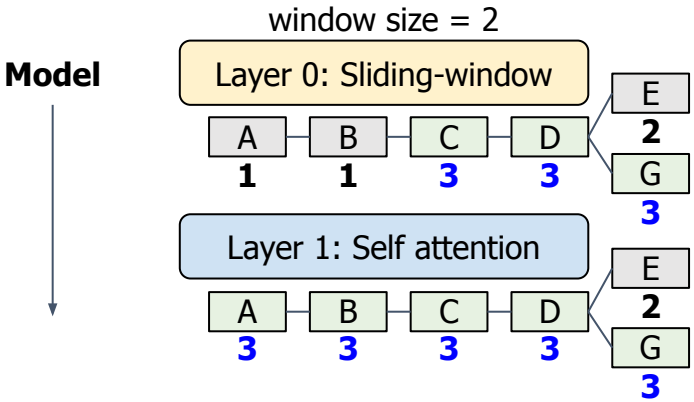


Fig 18. KV cache page with last-access time

Step	Request phase	Sequences
1	1's prefill	ABCD > E
2	1's decode	ABCDE > F
3	2's prefill	ABCDG > H

Fig 19. Timeline of the 2 requests

Customizable Eviction API

- **Balanced, aligned caching across heterogeneous layers**
  - *update_last_access*: Set unifies last-access times across different layers (**balanced eviction**)
  - *set_prefix_length*: Refine eviction priorities for pages (**aligned eviction**)

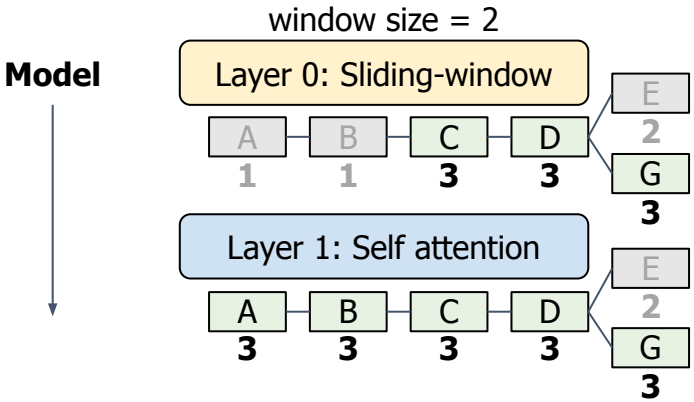


Fig 18. KV cache page with last-access time

Step	Request phase	Sequences
1	1's prefill	ABCD > E
2	1's decode	ABCDE > F
3	2's prefill	ABCDG > H

Fig 19. Timeline of the 2 requests

Customizable Eviction API

- **Balanced, aligned caching across heterogeneous layers**
  - *update_last_access*: Set unifies last-access times across different layers (**balanced eviction**)
  - *set_prefix_length*: Refine eviction priorities for pages (**aligned eviction**)

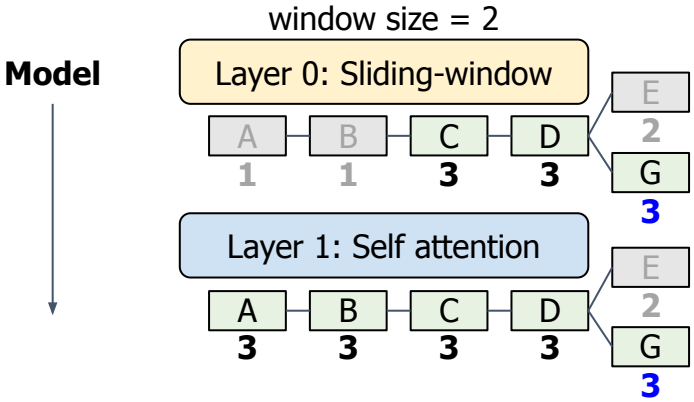


Fig 18. KV cache page with last-access time

Step	Request phase	Sequences
1	1's prefill	ABCD > E
2	1's decode	ABCDE > F
3	2's prefill	ABCDG > H

Fig 19. Timeline of the 2 requests

Customizable Hit API

- **Valid prefixes differ across heterogeneous layers**
  - *get_possible_prefix*: Custom the cache hit rules for different layer types
  - **The longest common prefix** valid across all layers is selected as the **cache hit prefix**.

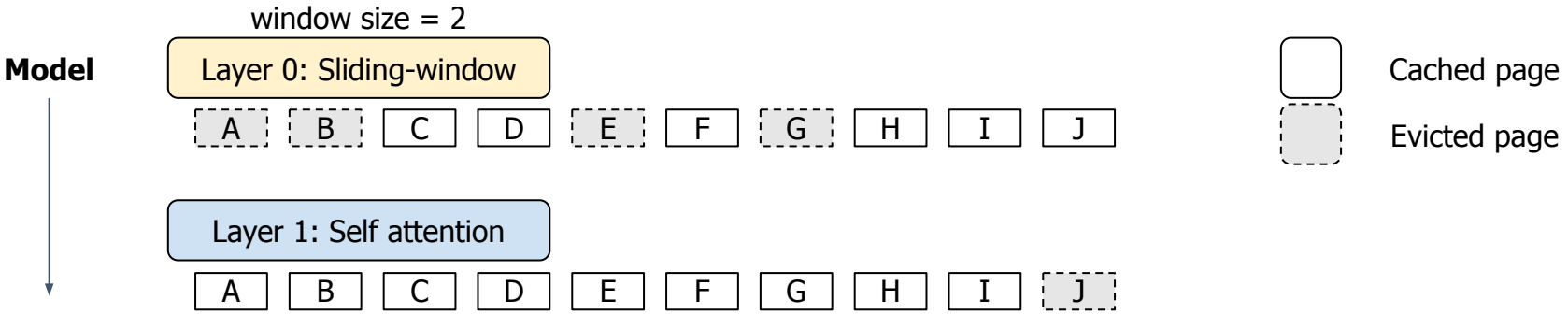


Fig 20. Example KV cache

Customizable Hit API

- **Valid prefixes differ across heterogeneous layers**
  - *get_possible_prefix*: Custom the cache hit rules for different layer types
  - **The longest common prefix** valid across all layers is selected as the **cache hit prefix**.

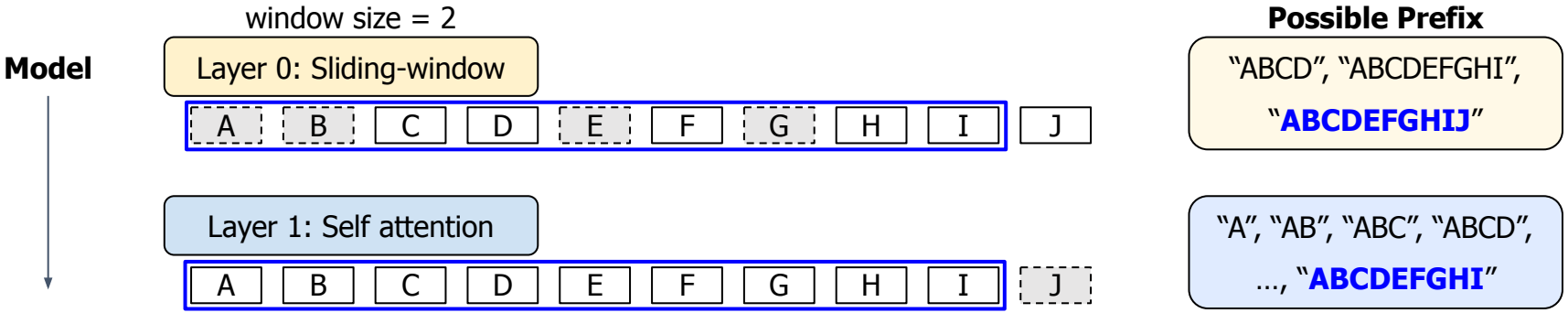


Fig 21. The possible prefixes for request "ABCDEFGHIJ"

Evaluation Setup

- **Baseline**
  - vLLM (SOSP'23): Paged attention
  - SGLang (NeurIPS'24): Radix attention
  - TGI (Huggingface): Naïve allocation
- **Hardware**
  - GPU: NVIDIA H100 80GB
  - CPU: 2 Intel Xeon Platinum 8480C
- **Model**
  - Multi-modal model with cross attention
    [Llama vision - mllama 11B]
  - Model with sliding window
    [Gemma-2 27B, Ministral 8B]
  - Model with Mamba layer
    [Jamba-1.5 52B]
  - Model with sliding window + KV cache sharing
    [Character.ai 70B]
  - Sparse attention
    [PyramidKV 70B]

Result 1. End-to-end Evaluation

- **Throughput**
  - JENGA turns less memory waste and better prefix reuse into real end-to-end throughput gains across diverse models.

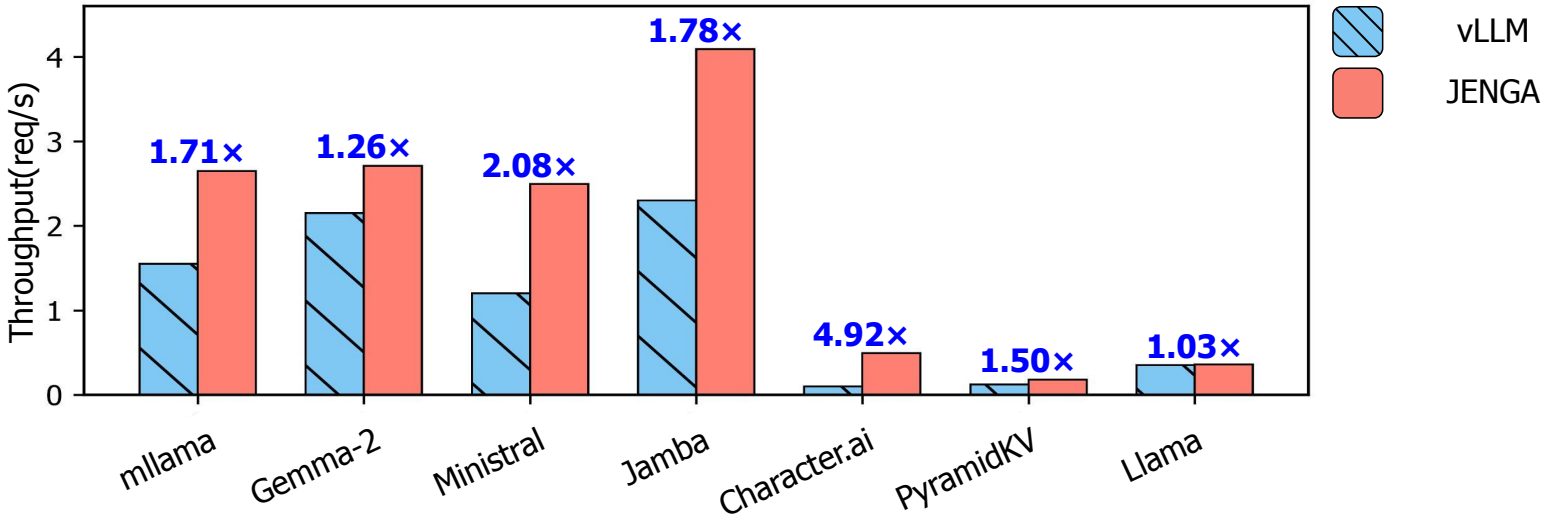


Fig 22. Inference throughput of vLLM and JENGA

Result 1. End-to-end Evaluation

- **Latency, Time to first token(TTFT), Time to output token(TPOT)**
  - JENGA improves latency under high load by freeing KV capacity for larger batching, without slowing down low-load cases.

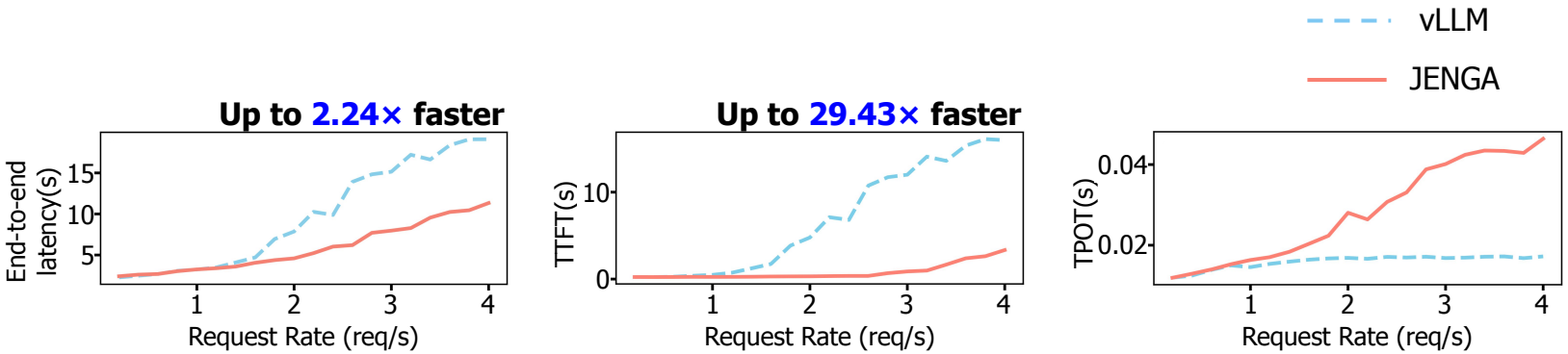


Fig 23. Latency under different request rates of multi-modal model with cross attention [mllama]

Result 2. Breakdown

- **Fragmentation analysis (Challenge 1 - Heterogeneous KV sizes)**
  - JENGA eliminates KV memory waste caused by heterogeneous KV sizes.

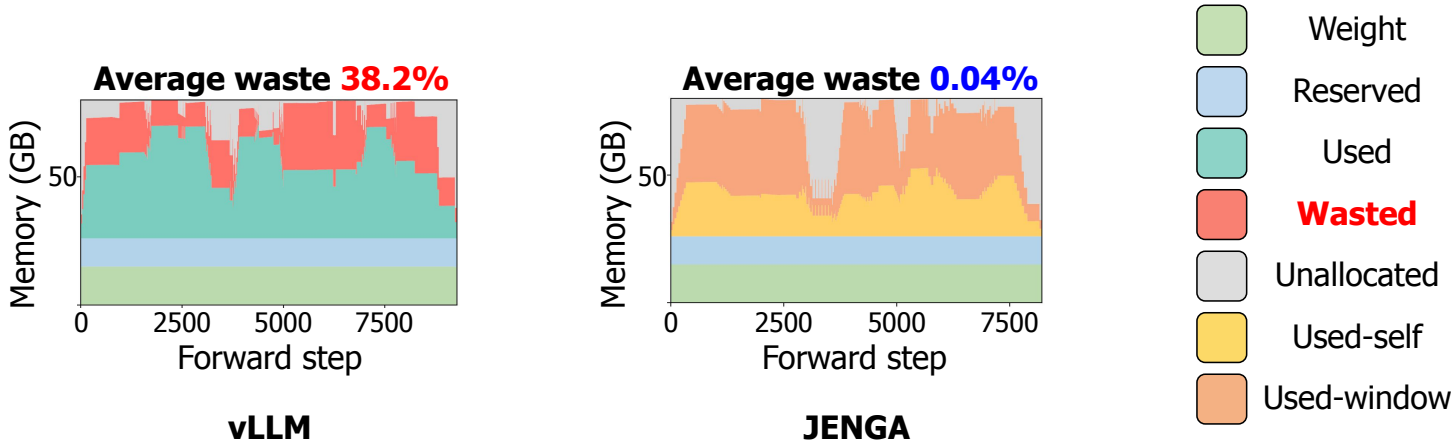


Fig 24. Timeline of memory usage for model with sliding window [Ministral]

Result 2. Breakdown

- **Prefix caching analysis (Challenge 2 - Heterogeneous Dependency)**
  - JENGA keeps prefix caching effective even when hit conditions differ across layer types.

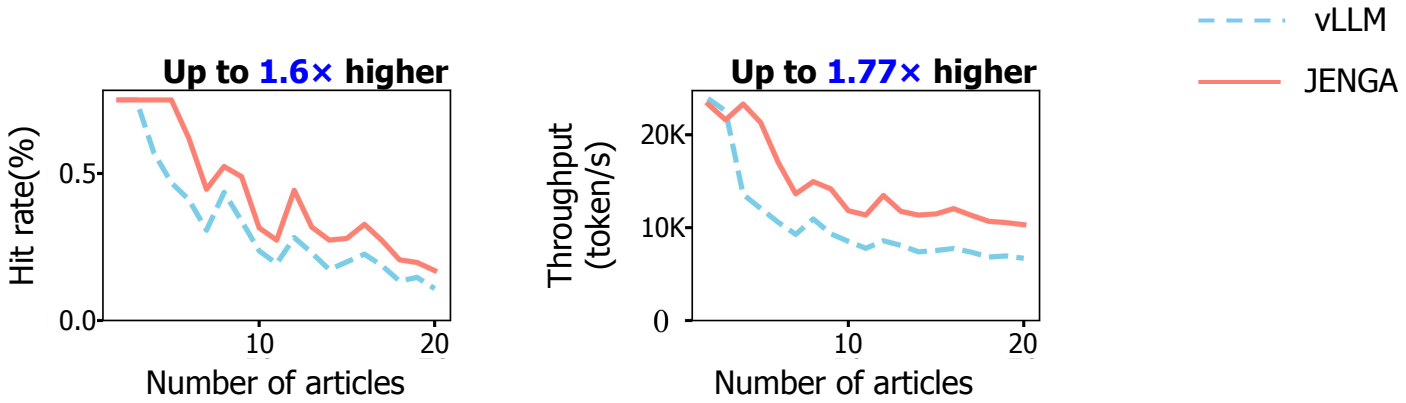


Fig 25. Prefix caching with different number of articles [arXiv-QA]

Conclusion

- **Recognize Challenge**
  - Identify the growing heterogeneity of new LLM architectures.
    (embedding sizes & token dependencies)
- **Propose novel & efficient system**
  - Propose a two-level memory allocation system for KV cache.
    (support different-size embeddings & provide flexibility to customize caching and eviction)
- **Implement prototype on top of vLLM**
  - Provide support for a wide range of optimizations and features.
- **Achieve superior serving performance**
  - Achieve a 1.80× increase in throughput over state-of-the-art LLM inference engines, without any impact on latency.